

DevOps Laboratory Work

Linux Git Docker and Jenkins

Student: Kazbek Assanbek

Group: IT2-2302

Student ID: 37765

Course: DevOps

Date: 8 October 2026

This report records the implemented and verified individual laboratory project, including publication to GitHub and a real local Jenkins pipeline execution.

Student Information

Field	Value
Name	Kazbek
Surname	Assanbek
Group	IT2-2302
Student ID	37765
Course	DevOps

The project directory is `assanbek_kazbek`. The repository, image, container, and Jenkins job names use the required personalized naming convention.

Linux

The required project folders `app`, `scripts`, `config`, `logs`, and `backup` were created. The information file contains the required student values. All Linux-specific command evidence was captured from an `aarch64` Linux Docker container, rather than from `macOS`.

The recorded session demonstrates `pwd`, `ls`, `cd`, `mkdir`, `touch`, `cp`, `mv`, `rm`, `cat`, `head`, and `tail`. It finds `application.log` with `find`; filters `ERROR` and `WARNING` messages with `grep`; counts two `ERROR` records; and displays the final five records.

Permissions were set to `600` for `private.txt` and `644` for `public.txt`. Read, write, and execute are represented by `r`, `w`, and `x`. The three permission groups are owner, group, and others. Private notes have owner-only access, while public information is readable by others.

The executable Bash script prints the student data, username, hostname, date, operating system, disk use, and memory use using `whoami`, `hostname`, `date`, `uname`, `df`, and `free`. It also confirms that the information file exists.

A process is a running program and its PID is a unique process identifier. `ps` provides a snapshot; `ps aux` adds owner, CPU, memory, and command details; `top` provides a refreshing view. The evidence records a Linux process table with all requested fields.

Git and GitHub

A Git repository was initialized on the main branch. The existing configured identity, Kazbek and 37765@iitu.edu.kz, was checked and not overwritten. The commit history contains six substantive commits and one merge commit: project initialization, the environment-configurable application, Linux verification, Docker verification, Compose lifecycle verification, documentation, and the merge.

The .gitignore file excludes *.log, backup/, and .env. The verification command git check-ignore -v confirmed that the application log, backup directory, and .env pattern are ignored. This prevents generated logs, local backups, and sensitive environment files from being committed.

A development branch was created, documentation was changed there and committed, then the branch was merged into main with a non-fast-forward merge. Branches allow work to be isolated and reviewed before integration.

The public repository was created and pushed at <https://github.com/RealKazbek/assanbek-kazbek-devops>. GitHub API checks confirmed the main branch and all required files.

Docker

The application is a small Python standard-library HTTP service. It prints the required student information and reads STUDENT_NAME, STUDENT_SURNAME, STUDENT_GROUP, and STUDENT_ID at runtime. Unit tests confirmed default values and an environment override.

The Dockerfile uses FROM python:3.13-alpine, WORKDIR /app, COPY for the application, RUN to create a non-root user, EXPOSE 8080, and CMD to launch the service. Each instruction has an operational purpose.

The image assanbek-kazbek-devops was built successfully on the local Linux/aarch64 Docker daemon. The named container assanbek-kazbek-container was run with the required environment variables. docker images, docker ps, docker ps -a, docker logs, docker exec, docker stop, docker start, and docker rm were exercised on laboratory resources.

Docker Compose

docker-compose.yml builds and starts the service with the required student environment values. docker compose up -d --build, compose ps, compose logs, and compose down completed successfully. Compose is useful because the configuration and lifecycle command are repeatable from a version-controlled file.

Environment variables are runtime configuration values passed to a process. They allow the same image to be configured in different environments. A real run with Changed Student, group DEMO-1, and ID 99999 showed changed application output. Sensitive values must not be hard-coded in an image because image layers and history may be shared or inspected.

Jenkins

A declarative Jenkinsfile is included with meaningful Checkout, Build, Test, Docker Build, and Docker Run stages. Checkout uses the configured SCM and records the selected commit. Build compiles Python source. Test runs unittest discovery. Docker Build creates the image. Docker Run starts a uniquely named container and checks its output.

A local authenticated Jenkins job named Assanbek_Kazbek_DevOps was created with the public GitHub repository as SCM. Build number 2 checked out commit e16e061, completed every required stage, displayed the student environment values, and ended with Finished: SUCCESS.

Integrated DevOps Workflow

Linux provides a consistent automation environment. Git records changes and creates controlled branches. GitHub hosts the remote source that Jenkins checks out. Jenkins then builds and tests the selected commit, packages it as a Docker image, and runs a container from that image. An image is the immutable package; a container is a running instance. If a pipeline stage fails, later stages do not run and Jenkins reports the failed result.

The entire Linux, Git, GitHub, Jenkins, Docker-build, Docker-run and Compose workflow was verified. Jenkins was local-only with authentication enabled and its API/console evidence was captured before the temporary controller was stopped.

Screenshots and Evidence

No fabricated screenshots are included. The following files contain genuine command output captured during the local Linux and Docker executions. They can be opened while recording the required continuous demonstration.

Evidence file	Contents
evidence/linux/linux_commands.txt	Linux kernel identity; commands; search; permissions; script; ps, ps aux and top.
evidence/docker/docker_verification.txt	Docker build, image list, named container, logs, exec, override, stop and start.
evidence/docker/compose_and_management.txt	Compose up, ps, logs, down, removal demonstration, final container.
evidence/gitignore_check.txt	Actual ignore-rule matches.

The GitHub repository and Jenkins dashboard/stage evidence was captured using terminal and authenticated APIs only, as required by the execution constraints. The raw GitHub and Jenkins records are stored in the evidence directory.

Conclusion

This project implements and verifies the complete Linux, Git, GitHub, Docker, Docker Compose, and Jenkins workflow. It has a personalized application, tests, meaningful history, a branch merge, a public repository, and a real successful pipeline. The remaining submission action is the student's own continuous voice recording.

Verification Summary

Area	Status
Linux tasks 1-5	PASS
Git tasks 6-9	PASS
GitHub task 10	PASS
Docker tasks 11-15	PASS
Jenkins tasks 16-20	PASS
Integrated task 21	PASS